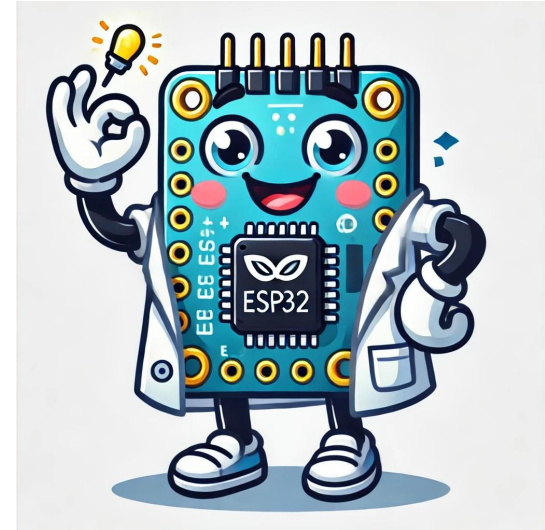




UNIVERSITÀ
degli STUDI
di CATANIA

ESP32: Configuration



davide.patti@dieei.unict.it

Hands-on ESP32: Simulators



Create an account at:

<https://wokwi.com/esp32>

- Most examples will be carried on the Wokwi simulator
- Directly applicable to the physical microcontroller with the Arduino IDE configured for running the ESP32



World's most advanced ESP32 simulator

[Discord Community](#)

[LinkedIn Group](#)

[Pricing](#)

Docs



Davide Patti

xedivad@gmail.com

HOBBY ACCOUNT

<> [My projects](#)

[Profile](#)

[CI Tokens](#)

[Your Subscriptions](#)

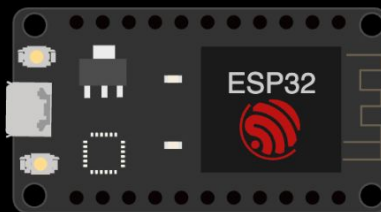
[Language](#)

[Logout](#)

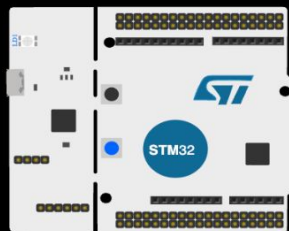
Simulate with Wokwi Online



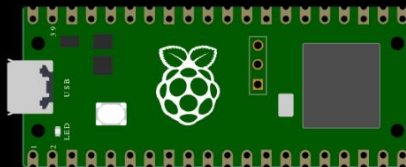
Arduino (Uno, Mega, Nano)



ESP32



STM32



Pi Pico

Wokwi Simulation: Remote & Shared



Executed Remotely:

Simulations run on powerful external servers.



Free Accounts Queue:

Wait time required (often a few seconds).

Due to shared resources.



Alternative Solution



Wokwi Extension for Visual Studio Code

- Configure for a local-like development experience, bypassing the remote queue.

Feature

Wokwi Simulator

Physical ESP32-DevKitC-32E

**WiFi
Connection**

Virtual Network: You cannot connect to your university's "Eduroam" or home WiFi. You must use the special SSID: `Wokwi - GUEST` with no password.

Real RF: Connects to actual SSIDs. Requires real credentials (SSID/Password).

**Execution
Speed**

Variable: I may run slower than real-time depending on your computer's CPU. Look at the simulation percentage (e.g., "80%") in the corner.

Real-Time: I always run at 240MHz (or configured speed).

**Analog
Inputs (ADC)**

Idealized: Inputs are controlled by sliders/mouse. No electrical noise or impedance issues.

Noisy: ADC pins are sensitive to noise and require filtering/averaging in code for stability.

**Serial
Monitor**

Instant: No baud rate mismatch issues. The simulator captures `Serial.print` instantly.

Config Dependent: Students must match the baud rate (e.g., 115200) in the monitor, or they will see garbage text.

**Pin
Constraints**

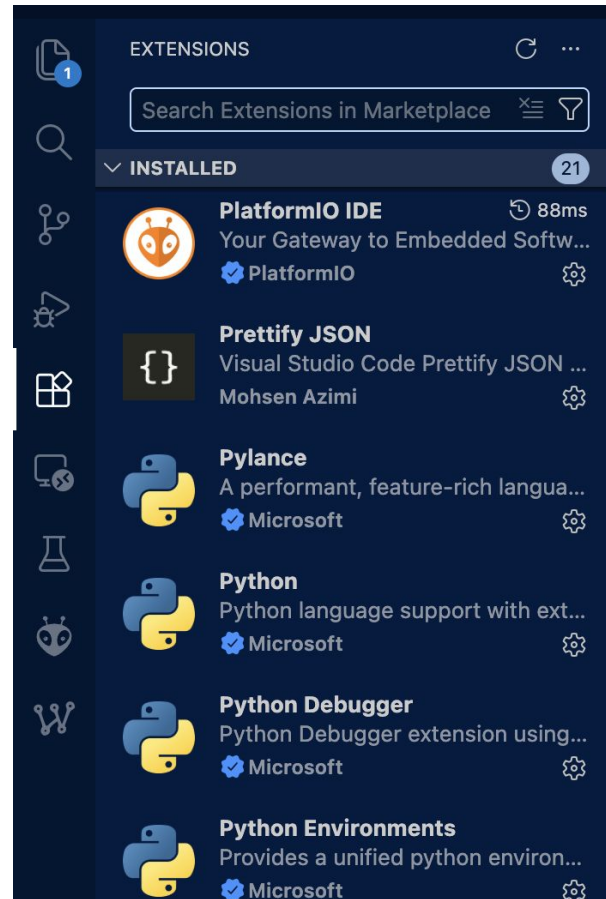
Permissive: I might let you draw too much current from a pin without "smoking."

Strict: Drawing >40mA from a GPIO will damage the physical chip.

VS Code ESP32 Simulator Setup

STEP 1. [ONLY THE FIRST TIME] Install Extensions: Open (Ctrl+Shift+X) and install:

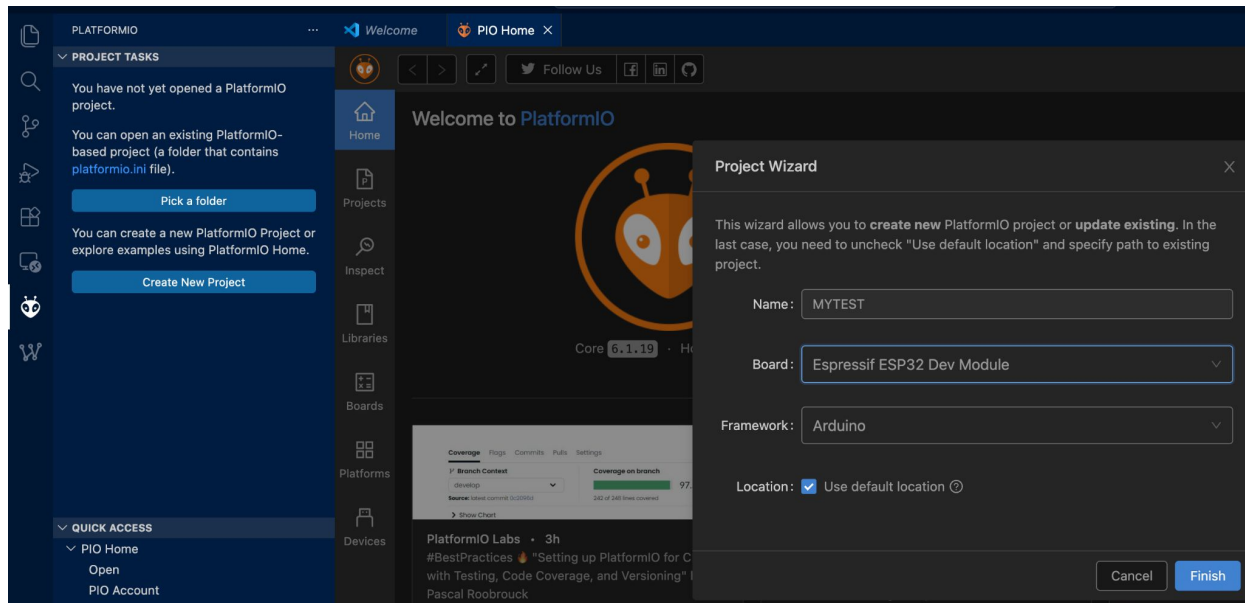
- **PlatformIO IDE** (by PlatformIO) – *The build system.*
- **Wokwi Simulator** (by Wokwi) – *The simulation engine.*



VS Code ESP32 Simulator Setup

STEP2: Create a New PlatformIO Project

- Click the **Alien Face icon** (PlatformIO) on the left sidebar.
- Click **"New Project"**
 - **Board:** Search for **"Espressif ESP32 Dev Module"**
 - **Framework:** Select **"Arduino"**.



VS Code ESP32 Simulator Setup

The 3 Core Project Files



`src/main.cpp`

This is the primary source file containing the C++ code to be executed on the ESP32.



`diagram.json`

(TO BE CREATED)

Defines and describes the hardware circuit and components in the Wokwi simulator.



`wokwi.toml`

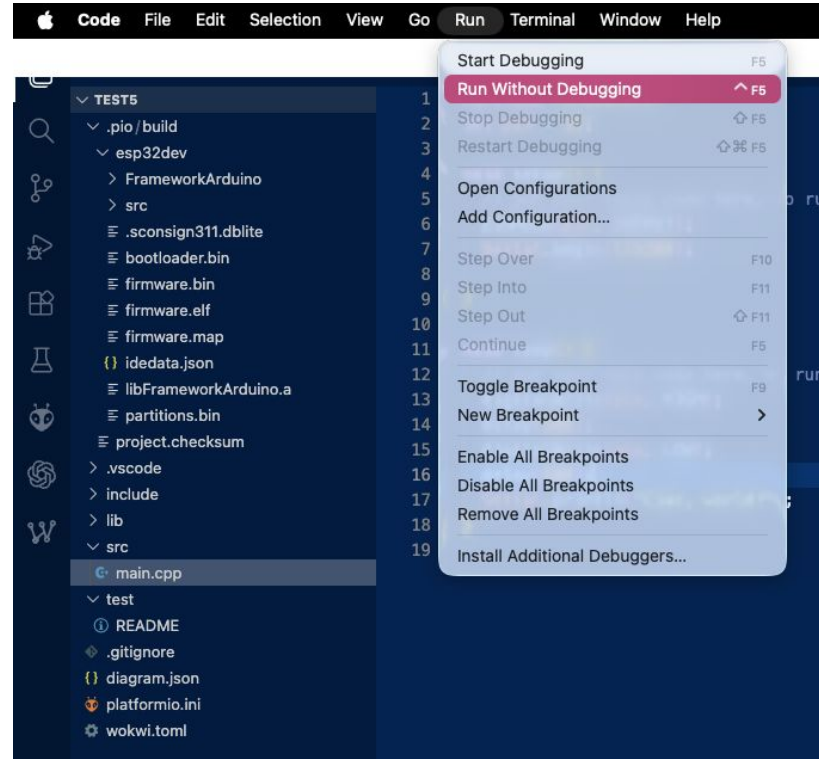
(TO BE CREATED)

Configuration file that specifies how the firmware is built and linked to the simulator.

Updating src/main.cpp

IMPORTANT: If you change the source code main.cpp, then you have to save and recompile it **BEFORE** running again the simulation (you must re-create the firmware!)

Click on the source file and then select the Run command from the menu on the top



Create diagram.json

Create an empty **diagram.json** in the root directory and paste the following base configuration for the **ESP32** microcontroller.

```
{
  "version": 1,
  "author": "Your Name",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-esp32-devkit-c-v4",
      "id": "esp32",
      "top": -201.6,
      "left": -302.36,
      "attrs": { "name": "ESP32" }
    }
  ],
  "connections": [],
  "dependencies": {}
}
```

Component Details

Board Type

[esp32-devkit-c-v4](#)

Purpose

Defines the virtual hardware environment and its spatial coordinates.

Configure wokwi.toml

Create a **wokwi.toml** file to specify the firmware location for simulation. **Remember to save!**

```
# Standard Configuration
[wokwi]
version = 1
firmware = '.pio/build/esp32dev/firmware.bin'
elf = '.pio/build/esp32dev/firmware.elf'
```

Configuration Info

PURPOSE

Links the compiled binary to the simulator.

Tip: Ensure the paths match your actual PlatformIO build directory.

Circuit Editing & Simulation Workflow



1. Design Circuit: diagram.json

Use the Wokwi graphic editor or copy from an existing project.

Note: Confirm the free license on first launch from Visual Studio Code



2. Edit Code: main.cpp

Modify your `main.cpp` directly in Visual Studio Code.

Ensure you recompile the source before starting simulation.



3. Simulate

Run the simulation to see your hardware and software interact.

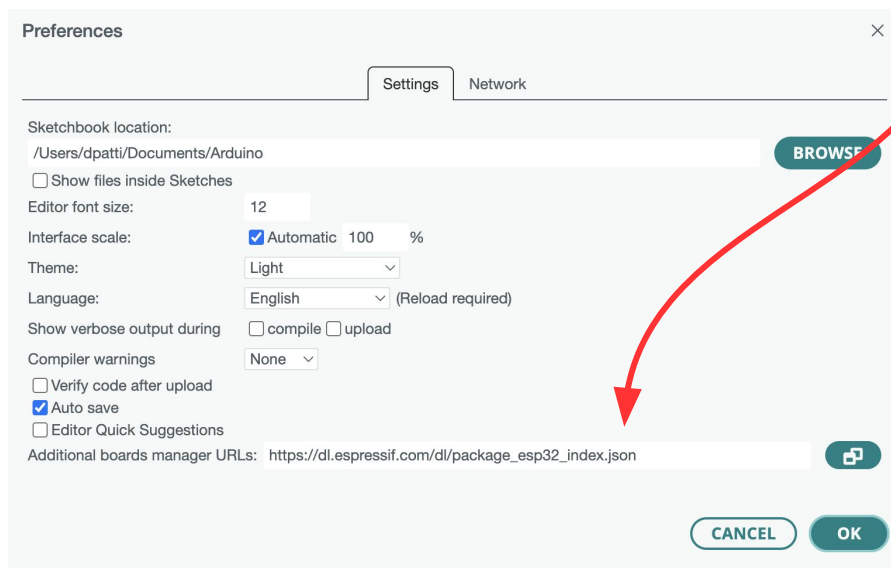
Tip: If the license fails, manually copy `diagram.json` from Wokwi web editor

Summary: VS Code for code editing • Wokwi for circuit design • Always recompile `main.cpp`

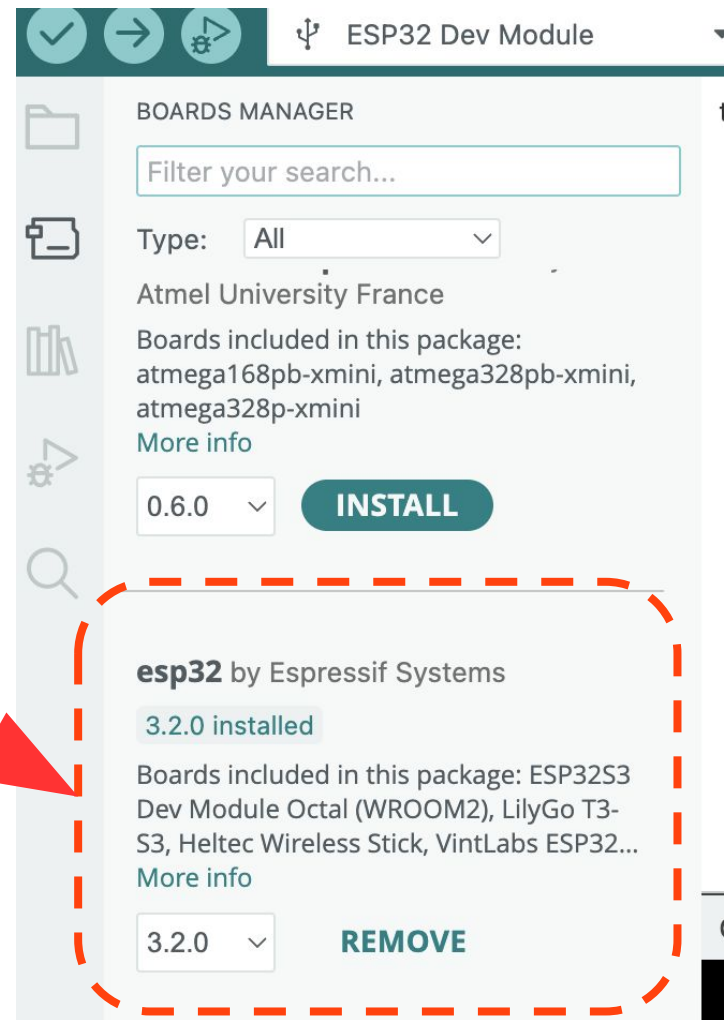
Configuring the ESP32 (Physical Device)

Configuring the Arduino IDE

- **The ESP32 is compatible with Arduino IDE:**
 - <https://www.arduino.cc/en/software>
- Go to Arduino IDE->Settings and add the following line as indicated below:
 - https://dl.espressif.com/dl/package_esp32_index.json



Install “esp32” by Espressif from BOARDS MANAGER



Installing Drivers (required on Windows)

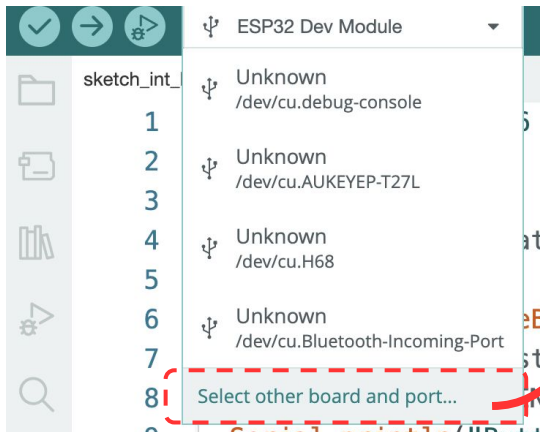
- **On Windows**, you will need to download the CP210x USB to UART Bridge VCP Drivers for your computer to be able to read the ESP32 board.
- To download them, go to:
<https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers?tab=downloads>
and select the driver for your computer type.

Software Downloads

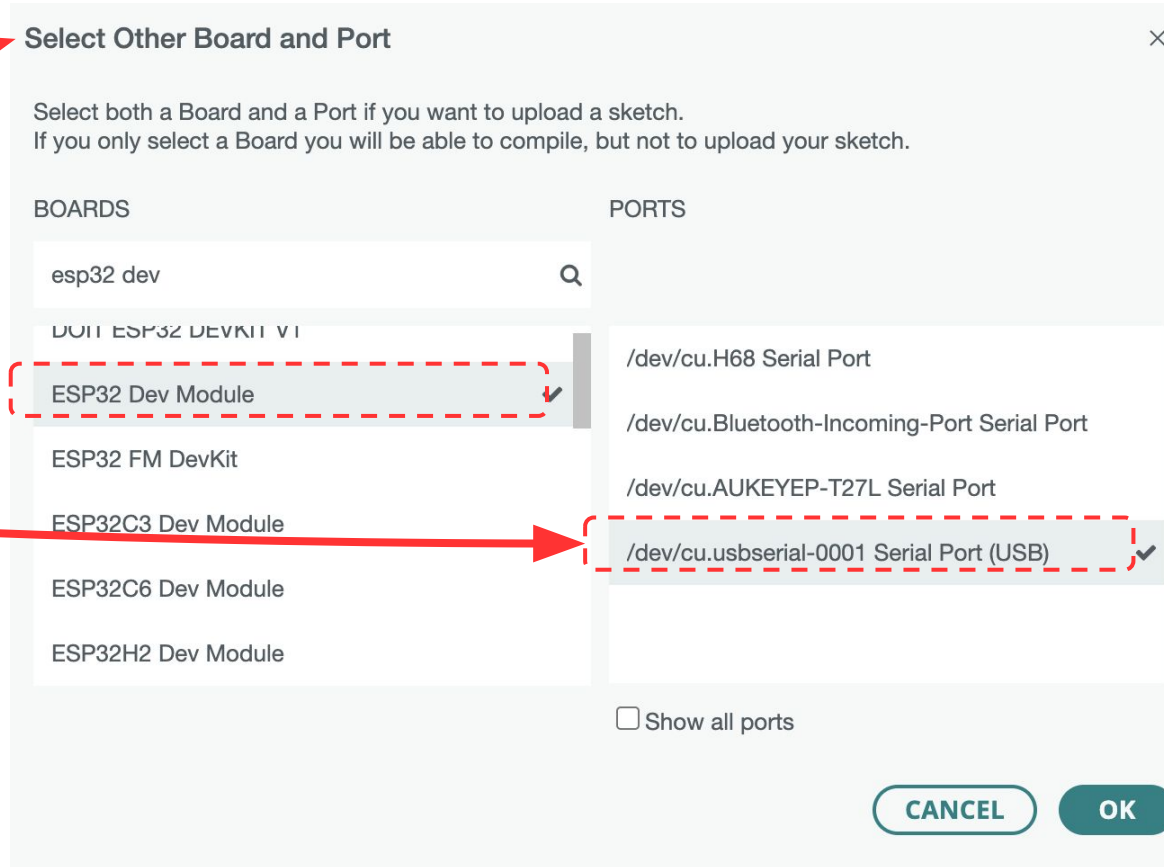
Software (11)

Software • 11

CP210x Universal Windows Driver	v11.4.0
CP210x VCP Mac OSX Driver	v6.0.2
CP210x VCP Windows	v6.7
CP210x Windows Drivers	v6.7.6
CP210x Windows Drivers with Serial Enumerator	v6.7.6



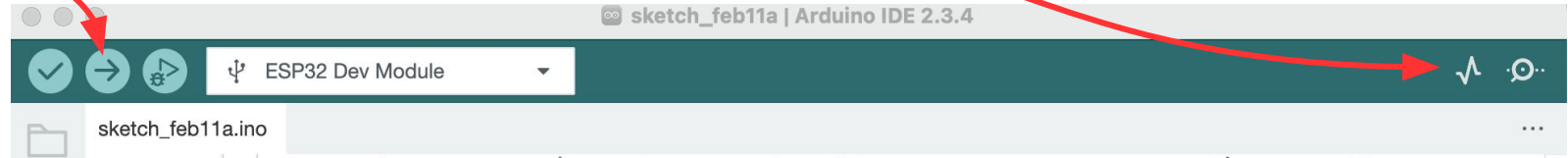
Final check: go in the menu and select the board



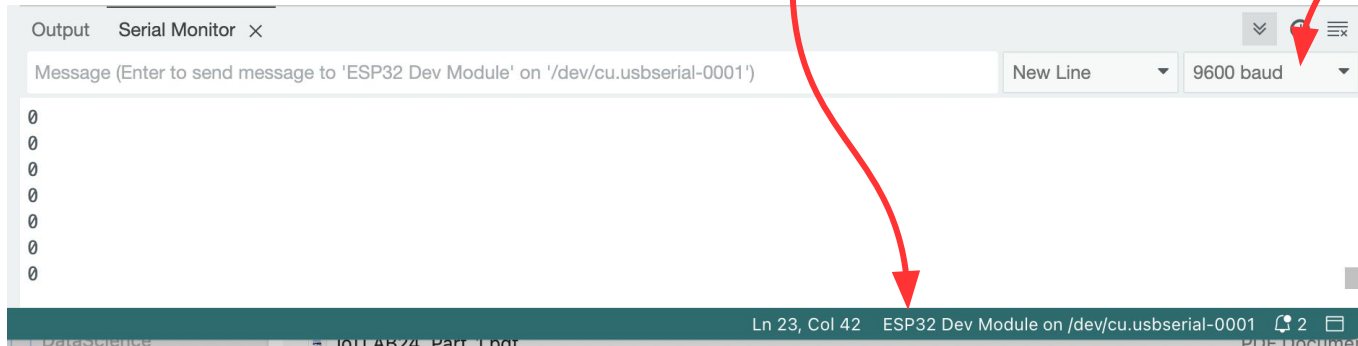
**Connect the board and
Select the USB port**

IMPORTANT

- **DO NOT DISCONNECT** the components while the board is connected!!
- Upload AGAIN of the program to the flash every time the program is changed
- Serial monitor is activated HERE



- Check that Serial monitor speed is THE SAME of the one of Serial.begin()
- Check that board is connected



Testing

- Upload a program:



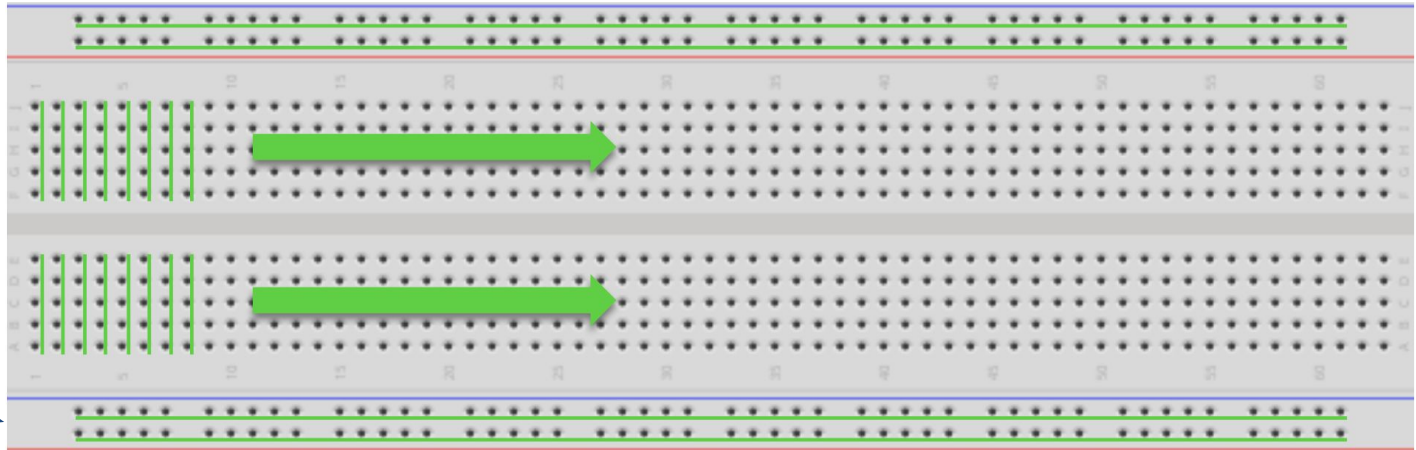
- Wait for compilation and uploading (a few seconds)

```
Flash will be erased from 0x00010000 to 0x00054fff...
Compressed 24976 bytes to 15972...
Writing at 0x00001000... (100 %)
Wrote 24976 bytes (15972 compressed) at 0x00001000 in 0.4 seconds (effective 514.3 kbit/s)
Hash of data verified.
Compressed 3072 bytes to 146...
Writing at 0x00008000... (100 %)
Wrote 3072 bytes (146 compressed) at 0x00008000 in 0.0 seconds (effective 587.2 kbit/s)
Hash of data verified.
Compressed 8192 bytes to 47...
Writing at 0x0000e000... (100 %)
Wrote 8192 bytes (47 compressed) at 0x0000e000 in 0.1 seconds (effective 882.9 kbit/s)
Hash of data verified.
Compressed 278896 bytes to 148601...
Writing at 0x00010000... (10 %)
Writing at 0x0001b7ff... (20 %)
Writing at 0x000281fd... (30 %)
```

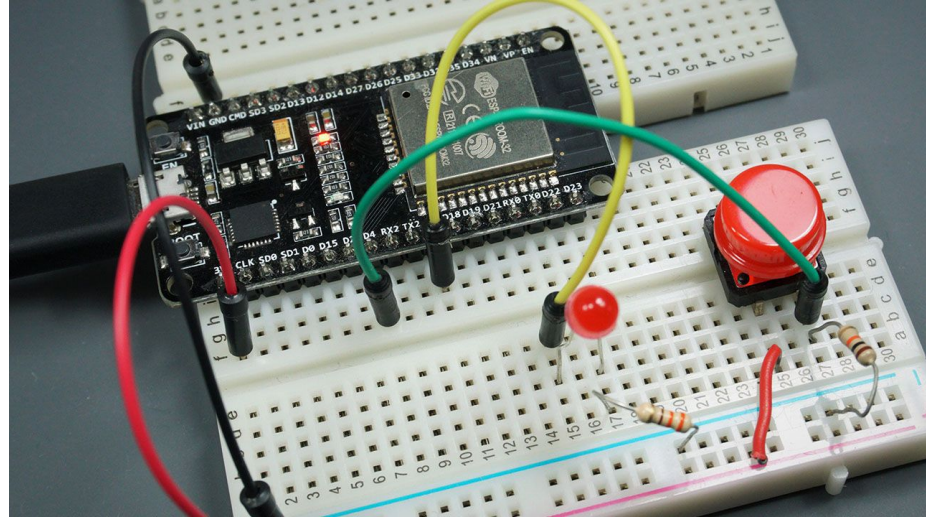
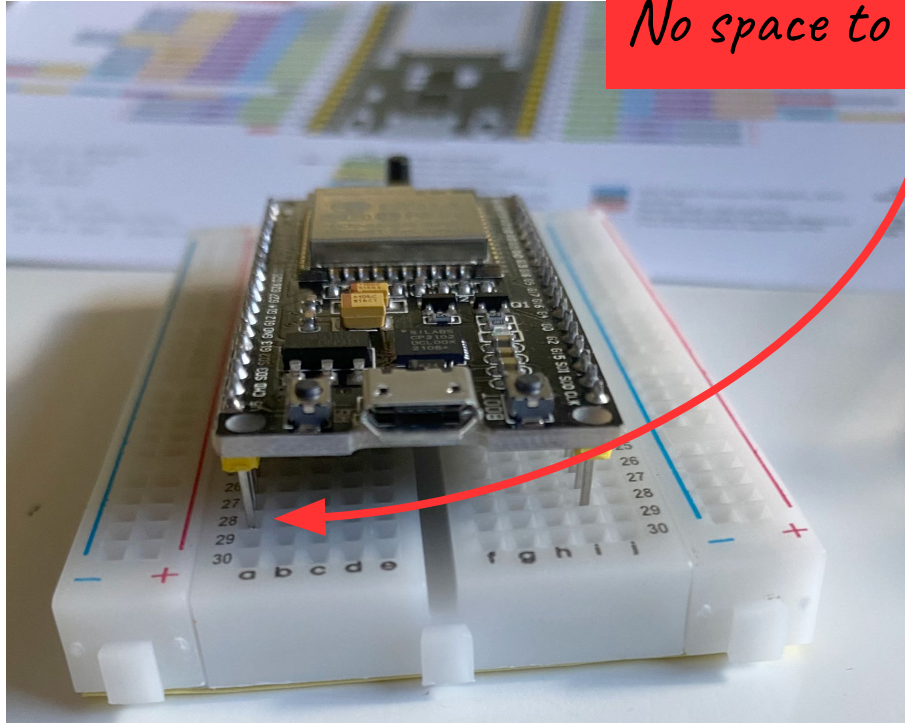
Breadboard

Horizontally connected lines, useful for voltage distribution (0, 3.3V)

- Useful for rapid prototyping
- Holes connected by green lines are short-circuited



No space to connect!



IMPORTANT: depending on the microcontroller size, you might need to arrange or split the breadboards!